SAGA LIGHT SOURCE | TECHNICAL SPECIFICATION & USER GUIDE

AILab_Note 2.0 (AIN2) Comprehensive Manual & Add-in Developer Guide

Digital Laboratory Notebook & Modular Add-in Specification for Synchrotron Radiation and Advanced Scientific Measurements

Document ID	SAGA-LS_AIN2_Manual_EN / Version 2.0.0 (Modular Edition)	**Copyright**	© SAGA Light Source (SAGA-LS)
Developer	SAGA Light Source (Chief Architect: Akio Yoneyama)	**AI Partners**	Emma, Emu & Eli

Table of Contents

- **Chapter 1. System Overview and Design Philosophy**
  - 1.1 Purpose and Core Concepts of AIN2
  - 1.2 System Architecture
  - 1.3 Multi-Account Profile Isolation
- **Chapter 2. Installation and Environment Setup**
  - 2.1 Recommended System Requirements
  - 2.2 Step-by-Step Setup Instructions (macOS / Windows 11)
  - 2.3 Startup and Single-Instance Locking
  - 2.4 Shutdown Sequence, Unsaved Detection, and Differential Backup
- **Chapter 3. Main Notebook Features and UI Operations**
  - 3.1 Screen Layout and Date Navigation
  - 3.2 Auto-Save and Multi-Layer Protection (.tmp / .bak / Differential Backup)
  - 3.3 Synchrotron Radiation Support Tools (SR Info & Ring Current)
  - 3.4 Rich Editor Formatting and Inline Canvas Drawing
  - 3.5 Log Search (Find Logs)
  - 3.6 Persistent Scratchpad (Memorandum)
  - 3.7 Scratchpad Calculator
  - 3.8 Fiji / ImageJ Bidirectional Integration
  - 3.9 Bilingual UI (i18n) and Preferences (Option)
- **Chapter 4. Modular Add-in Suite (AIN2 Add-ins)**
  - 4.1 Bookmarks and Links Manager (Links - 01_links)
  - 4.2 Standard Protocol Manager (02_protocol_manager)
  - 4.3 Telegram Field Notes & Photo Capture (COP - 03_cop_telegram)
  - 4.4 AI Research Partner "Eli" (04_eli_assistant)
- **Chapter 5. System Architecture & Internal Specification Reference**
  - 5.1 Directory Structure and File Layout
  - 5.2 Core Backend API Endpoints
  - 5.3 Generic Inter-Window Communication (BroadcastChannel)

- **Chapter 6. Add-in Developer & Plugin Specification Guide**
  - 6.1 Add-in Structure Requirements and Lifecycle
  - 6.2 manifest.json Specification Reference
  - 6.3 Frontend Communication Protocol (ain_addin_bridge)
  - 6.4 Backend API Router Specification (router.py)
  - 6.5 Minimal Add-in Implementation Tutorial

Chapter 1. System Overview and Design Philosophy

1.1 Purpose and Core Concepts of AIN2

AILab_Note 2.0 (AIN2) is a hybrid digital laboratory notebook designed for demanding experimental environments—such as synchrotron radiation beamlines (X-ray CT, XAFS, micro-diffraction, and scattering)—where parameters and image datasets evolve rapidly in real time.

- **Complete Local Self-Sufficiency:** Operates with millisecond-level responsiveness even inside radiation-shielded experimental hutches or air-gapped networks without internet access.
- **Pluggable Modular Add-in Architecture:** Decouples the core notebook engine from specialized tools (AI assistant, mobile field capture, standard protocols), allowing modules to be added, removed, or distributed independently.
- **Multi-Account Profile Isolation:** Supports up to five completely isolated user profiles on a single shared beamline workstation.

1.2 System Architecture

AIN2 integrates a Python (FastAPI) backend with a `pywebview` native desktop shell. Standard web technologies (HTML5 / CSS / Vanilla JS) communicate with a local HTTP/REST API (`127.0.0.1:8000`), combining low-latency UI responsiveness with robust file I/O and external scientific tool integration.

1.3 Multi-Account Profile Isolation

In shared beamline facilities and collaborative laboratories, multiple researchers and experimental groups rotate on the same control PC. Users can switch between up to five accounts instantaneously via the account selector in the top-right toolbar. All daily logs, configuration files (`config.json`), ChromaDB vector indices, and cached Telegram photos are stored independently under `./Log/{account_name}/` .

Chapter 2. Installation and Environment Setup

2.1 Recommended System Requirements

Component	macOS Recommended Spec	Windows 11 Recommended Spec
OS	macOS 13 (Ventura) or later (Apple Silicon / Intel)	Windows 11 64-bit (22H2 or later)
Python	Python 3.10 or 3.11 (Conda environment recommended)	Python 3.10 or 3.11
RAM / Display	16 GB or more recommended | 1920×1080 or higher	16 GB or more recommended | 1920×1080 or higher
External Tools	Fiji (ImageJ.app), Tesseract OCR (via Homebrew)	Fiji (ImageJ-win64.exe), Tesseract OCR

2.2 Step-by-Step Setup Instructions (macOS / Windows 11)

A. macOS Setup Procedure

1. **Install Homebrew and Tesseract OCR:** Open Terminal and install Homebrew along with Tesseract OCR (used for reading storage ring current monitors).

```
# Install Homebrew (if not already installed)
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install Tesseract OCR
brew install tesseract
```

2. **Install Miniconda and Create Virtual Environment:**

```
# Create and activate the 'ain2' Python 3.11 environment
conda create -n ain2 python=3.11 -y
conda activate ain2
```

3. **Install Python Dependencies:** Navigate to the AIN2 root directory and install the required packages.

```
cd /path/to/AIN2
pip install fastapi uvicorn pydantic pywebview beautifulsoup4 requests opencv-python
numpy pytesseract xraylib chromadb
```

> **Note on pywebview under macOS**
> On macOS, `pywebview` directly uses the system-native WebKit (Safari) engine, eliminating the need for heavy external GUI frameworks such as Qt.

B. Windows 11 Setup Procedure

1. **Prepare System Prerequisites:**
   - **Tesseract OCR (Windows):** Download `tesseract-ocr-w64-setup.exe` from the UB-Mannheim repository, install it to `C:\Program Files\Tesseract-OCR`, and add that directory to the system `Path` environment variable.
   - **Microsoft C++ Build Tools:** May be required when building scientific libraries such as `xraylib`. Install via the official Visual Studio Installer if needed.
2. **Create Conda Environment via Anaconda Prompt:**

```
conda create -n ain2 python=3.11 -y
conda activate ain2
cd C:\path\to\AIN2
pip install fastapi uvicorn pydantic pywebview beautifulsoup4 requests opencv-python
numpy pytesseract xraylib chromadb
```

> **Note on pywebview under Windows 11**
> Windows 11 includes the Microsoft Edge WebView2 runtime out of the box, enabling crisp HTML5 rendering without additional runtime installations.

2.3 Startup and Single-Instance Locking

With the virtual environment activated, launch the application via `python main.py`. To prevent file corruption from accidental duplicate launches, AIN2 acquires an exclusive lock on `.ain_app.lock` in the project root (`fcntl.flock` on macOS; `msvcrt.locking` on Windows). If an instance is already running, the OS brings the existing window to the foreground and the duplicate process exits cleanly.

2.4 Shutdown Sequence, Unsaved Detection, and Differential Backup

When closing the main window, AIN2 intercepts the `pywebview` closing event and checks the `isModified` flag. If unsaved changes exist, a confirmation dialog prompts the user to "Save & Exit", "Discard & Exit", or "Cancel". Upon exit, temporary files in `tmp_imagej/` are purged, and if configured, an incremental backup (`rsync --update`) mirrors log data to a designated local or cloud-synced directory (such as Dropbox).

Chapter 3. Main Notebook Features and UI Operations

3.1 Screen Layout and Date Navigation

The main interface consists of four zones: the left-side **Tool & Add-in Launcher Bar**, the top **Formatting & Action Header**, the central **Rich Editor Canvas (`contenteditable`)**, and the bottom **Status Bar**. Use `◀ / ▶` in the header to step backward or forward by one calendar day, or `◀◀ / ▶▶` to jump directly to the previous or next date containing recorded log entries. A visual calendar picker is also available for direct date selection.

3.2 Auto-Save and Multi-Layer Protection (.tmp / .bak / Differential Backup)

Active edits are asynchronously written to a `.tmp` file after a short typing debounce (~1.5 seconds). During manual saves or periodic auto-saves, the existing `YYYYMMDD.html` file is rotated to `YYYYMMDD.bak` before the new file is atomically replaced on disk. Enabling differential backup in Option triggers automatic synchronization to an external backup path on application exit.

3.3 Synchrotron Radiation Support Tools (SR Info & Ring Current)

- **Experimental Conditions Table (SR Info):** Opens a parameter dialog to record Facility (SAGA-LS, SPring-8, PF), Beamline, Monochromator Crystal / Filter, X-ray Energy (keV), Measurement Method, and Participants, inserting a formatted metadata table at the cursor position.
- **Storage Ring Current Acquisition (Ring Current):** Fetches the status monitor image from the configured synchrotron facility in the background, extracts the beam current (mA) via built-in OCR (OpenCV + Tesseract), and inserts the value with a timestamp.

3.4 Rich Editor Formatting and Inline Canvas Drawing

1. Text Formatting Tools:

- One-click toggles for **Bold (B)**, *Italic (I)*, Underline (U), ~~Strikethrough (S)~~, Superscript (x^2), and Subscript (x_2).
- Text color toggle (default black vs. alert red) and yellow highlighter marker.
- Bulleted lists (`<ul>`), numbered lists (`<ol>`), dashed horizontal dividers, and bordered callout boxes.

2. Inline Canvas Drawing & Image Annotation:

Clicking the canvas button opens a line-height selector and inserts a blank drawing canvas into the editor. Clicking any image inside the notebook activates a blue resize frame and a slim annotation toolbar:

- **Line & Arrow Tool:** Line thickness slider (1–10 px), stroke style (solid, dashed, dash-dot), arrowheads (none, right →, left ←, bidirectional ↔), and custom color picker.
- **Shape Tools (Rectangle / Circle & Ellipse):** Border width, stroke style, border color, fill hatching (none, solid fill, diagonal hatch, cross-grid), and fill color.
- **Text Annotation Tool (T):** Places custom text labels at clicked coordinates with configurable font size (8–48 px), bold, italic, underline, and color.
- **Undo / Redo & Commit Protection:** Supports full undo/redo history while drawing. Clicking "Commit" (or selecting another image) rasterizes vector overlays permanently into the underlying Base64 image at native resolution. Uncommitted edits trigger a protective confirmation prompt before switching context.

3.5 Log Search (Find Logs)

Accessible from the search icon on the left sidebar:

- **Full-Text Keyword Search:** Strips HTML tags and Base64 image payloads automatically to search pure text across all historical daily logs.
- **Date Range Filter:** Enable the "Date range" checkbox to restrict queries to a specific beamtime run (e.g., 2026-05-01 to 2026-05-06).
- **Context Preview & Jump:** Displays matching snippets with highlighted keywords; double-clicking any result loads that day's log immediately into the editor.

3.6 Persistent Scratchpad (Memorandum)

An independent sub-window for year-round notes such as reagent inventories, emergency contacts, and beamline optical constants that are independent of daily log dates.

- **Independent Storage:** Saved to `./Log/{account}/memorandum.html` so it remains persistent across date navigation.
- **Dual Backup Rotation:** Copies the previous state to `memorandum.orig` at startup and rotates `memorandum.bak` on every save.
- **Quick Formatting:** Includes interactive checkboxes, lists, bold text, and red alert text toggles for lightweight task tracking.

3.7 Scratchpad Calculator

Launched from the calculator icon on the left sidebar, combining a free-form scratchpad editor with a scientific and X-ray optics evaluation engine.

- **Input Syntax Rules:**
  - **Inside Scratchpad Calculator:** Enter expressions directly without a prefix (e.g., `180000 div 360` or `e2ang(10, si111)`) and press Enter to output the result on the next line.
  - **Inside Main Notebook:** Prefix expressions with `?` (e.g., `?Fe(10)` + Enter) to evaluate inline without interfering with normal prose.
- **X-ray Optics & Bragg Diffraction Functions:**
  - `e2w(E)`: Converts X-ray energy (keV) to wavelength (nm).
  - `w2e(lambda)`: Converts wavelength (nm) to X-ray energy (keV).
  - `e2ang(E, crystal)`: Calculates the Bragg angle (deg) for a given energy and crystal reflection (`si111`, `si220`, `si311`, `ge111`).
  - `ang2e(deg, crystal)`: Calculates the diffracted X-ray energy (keV) from a Bragg angle.
- **Elemental X-ray Absorption (xraylib):** Entering `Fe(10)` or `Si(17.8)` outputs linear attenuation coefficients, 1/e attenuation lengths, and absorption edge energies (e.g., `Fe@10keV: μ=221.73mm$^{-1}$, 1/e=0.0045mm, K-edge=7.112keV`).
- **Built-in Cheat Sheet (`??`):** Typing `??` and pressing Enter inserts a quick-reference sheet covering pulse-motor integer arithmetic (`div`, `mod`), prime factorization (`factor()`), trigonometry, and crystal constants.

3.8 Fiji / ImageJ Bidirectional Integration

Provides seamless inter-process communication (IPC) between AIN2 and Fiji/ImageJ for real-time X-ray CT and microscopy image analysis.

- **Automatic Launch & Background Daemon:** Selecting an image in the notebook and clicking the green microscope icon exports a temporary PNG and launches Fiji. Concurrently, a Jython daemon thread (`ain_watcher.py`) monitors Fiji's Macro Recorder and active window (`WindowManager.getCurrentImage()`) at 1-second intervals.
- **One-Click Result & Macro Import:** After performing contrast adjustments, scale-bar overlays, or particle analysis in Fiji, right-clicking inside the AIN2 editor and selecting "Img from Fiji" inserts both the processed image and the recorded ImageJ macro commands (e.g., `run("Enhance Contrast...", "saturated=0.3");`) into the log.

3.9 Bilingual UI (i18n) and Preferences (Option)

Click the gear icon at the bottom-left to open Preferences (Option). Switching the language between "Japanese" and "English (US)" updates all UI labels and tooltips in real time without restarting via a DOM `TreeWalker` and `ain_i18n_channel` synchronization across all open windows. Log paths, Fiji executable paths, auto-save intervals, backup destinations, and storage ring URLs are also configured here.

Chapter 4. Modular Add-in Suite (AIN2 Add-ins)

Specialized workflows are packaged as independent modules under the `Addins/` directory. Users and administrators can customize available features simply by adding or removing add-in folders without modifying the core codebase.

4.1 Bookmarks and Links Manager (Links - 01_links)

Manages frequently accessed web resources such as beamline status portals, radiation safety portals, and data analysis repositories.

- **Features:** Add, edit, and delete categorized bookmarks; clicking a link launches the OS default web browser.
- **Self-Contained Storage:** Stores data in its own `links.json` file inside the add-in folder without touching core databases.
- **Developer Starter Template:** Consisting solely of `manifest.json` and `window.html`, it serves as the minimal reference skeleton for third-party add-ins.

4.2 Standard Protocol Manager (02_protocol_manager)

Stores and organizes optical alignment procedures, sample-changer workflows, and calibration checklists as reusable step-by-step cards.

- **Step Card Editor:** Create, reorder, and save multi-step procedures (Step 1, Step 2, ...) isolated per user account.
- **One-Click Notebook Insertion:** Clicking a protocol dynamically generates a styled HTML checklist card (with interactive checkboxes and a blue accent border) and inserts it at the notebook cursor via `ain_addin_bridge`.

4.3 Telegram Field Notes & Photo Capture (COP - 03_cop_telegram)

Implements a Common Operational Picture (COP) bridge that pulls sample photos and quick text memos sent from smartphones inside the experimental hutch directly into AIN2 without cables.

1. **Create a Telegram Bot (@BotFather):**
   - Open Telegram, search for `@BotFather`, and send `/start` followed by `/newbot`.
   - Choose a display name and a unique username ending in `bot`, then copy the issued HTTP API Token (e.g., `7123456789:AAFnX...`).
2. **Obtain Authorized User IDs (@userinfobot):** Message `@userinfobot` on Telegram to retrieve your numeric user ID (e.g., `123456789`) so unauthorized messages are ignored.
3. **Configure in AIN2:** Open the gear icon inside the COP add-in panel, enter your `Bot Token` and comma-separated `Allowed User IDs`, and save.
4. **Field Workflow:** Send photos or notes to your bot from the beamline. Opening the COP add-in displays a 7-day reverse-chronological feed; clicking any card inserts the photo (resized to 380 px width with caption) or note directly into the notebook and marks the card as imported.

4.4 AI Research Partner "Eli" (04_eli_assistant)

An AI research assistant supporting Google Gemini 3.8 Flash and local vLLM/Ollama servers, organized into three primary tabs: **Chat**, **Log Formatting (eLog)**, and **RAG Generation**.

- **Multi-Model OpenAI-Compatible Connectivity:** Connects seamlessly to cloud endpoints (Google AI Studio Gemini 3.8 Flash) or local GPU inference servers (vLLM / Ollama). Clicking "Fetch Models" populates available model IDs directly from the target endpoint.
- **Chat Modes (Multi-Turn, Single-Turn, and Selection Quote):**
  - **Multi-Turn Context:** Retains the last six conversation exchanges by default for iterative experimental troubleshooting.
  - **Notebook Selection Quoting:** Highlighting text in the main notebook before opening Eli automatically attaches the excerpt as a quoted context block for targeted Q&A.
  - **Direct Insertion:** Clicking "Insert into AIN Note" below any response writes a formatted insight card directly into the active log.
- **Structured eLog Generation:** Converts raw daily HTML logs into clean YAML-frontmatter Markdown (`eLog`) structured into *Experimental Conditions*, *Procedure & Operation Log*, and *Discussion & ToDo*.
  - **View Modes:** Switch between "Log: Source" and "Log: Images Stripped" (which replaces heavy Base64 payloads with `[Image]` placeholders).
  - **Dual Formatting Workflows:** Format automatically in one click using the built-in Eli model, or copy the stripped text into an external web LLM (such as web Gemini or Emma), paste the resulting Markdown back into the editor pane, and save to `./eLog`.

- **Local ChromaDB RAG Knowledge Base:** Embeds saved `eLog/{account}/*_elog.md` files into a persistent local ChromaDB vector store.
  - **Incremental Sync:** Appends only newly created eLog dates in seconds at the end of each beamtime shift.
  - **Full Rebuild:** Re-indexes the entire historical archive cleanly whenever chunking rules or prompts are updated.
  - **Automatic Context Retrieval:** Queries relevant historical experiment chunks during chat sessions and injects them into Eli's prompt automatically.

Chapter 5. System Architecture & Internal Specification Reference

5.1 Directory Structure and File Layout

```
AIN2/
├── main.py                      # Application entry point, pywebview window manager,
instance lock, backup
├── core/                        # Core infrastructure modules
│   ├── __init__.py
│   ├── addin_loader.py          # Dynamic Addins/ directory scanner and router loader
│   └── config.py                # Per-account configuration CRUD and window geometry
persistence
├── routers/                     # Core backend FastAPI routers
│   ├── logs.py                  # Daily logs (.html/.tmp/.bak), full-text search,
memorandum API
│   ├── imagej.py                # Fiji IPC bridge and temporary image synchronization API
│   └── tools.py                 # Ring current OCR and native OS file/folder picker
dialogs
├── templates/                   # Core HTML templates (index.html, memo.html, calc.html)
├── static/                      # Static assets (css/, icon/, img/, js/)
│   └── js/                      # ain_main.js, ain_i18n.js, ain_calc_engine.js,
ain_canvas.js
├── Addins/                      # Independent modular add-in packages
│   ├── 01_links/                # Bookmarks and links manager
│   ├── 02_protocol_manager/     # Standard protocol manager
│   ├── 03_cop_telegram/         # Telegram field capture (COP)
│   └── 04_eli_assistant/        # AI research partner (Eli)
└── Log/{account}/               # Isolated per-account experimental data storage
```

5.2 Core Backend API Endpoints

Endpoint	Method	Description & Behavior
`/api/logs/{date_str}`	GET / POST	Retrieves or atomically saves raw HTML log for `YYYYMMDD` (with `.bak` rotation).
`/api/logs_dates`	GET	Returns an array of `YYYYMMDD` strings for all dates containing saved logs.
`/api/search`	POST	Executes full-text keyword search with optional start/end date range filtering.
`/api/memo`	GET / POST	Reads or writes the persistent scratchpad (`memorandum.html`).

Endpoint	Method	Description & Behavior
`/api/config`	GET / POST	Reads or updates `config.json` for the currently active user account.
`/api/accounts`	GET / POST	Lists available profiles, switches active account, or creates a new profile.
`/api/set_window_title`	POST	Updates the native OS window title bar for detached `pywebview` sub-windows.

5.3 Generic Inter-Window Communication (BroadcastChannel)

The main notebook editor and add-in windows communicate asynchronously via the HTML5 `BroadcastChannel` API. The core editor remains completely agnostic of specific add-in internals, exchanging structured HTML or plain text through standardized message bridges.

Channel Name	Primary Actions	Payload & Behavior
`ain_addin_bridge`	`insert_html` `insert_text` `request_selection`	Inserts formatted HTML or plain text from an add-in at the main editor cursor, or requests the currently highlighted text selection from the notebook.
`ain_i18n_channel`	`change_lang`	Broadcasts `{ action: 'change_lang', lang: 'ja' | 'en' }` to synchronize UI language across all open windows in real time.

Chapter 6. Add-in Developer & Plugin Specification Guide

Developers can package custom Python hardware drivers, data-acquisition scripts, or specialized web interfaces into official AIN2 add-ins using a straightforward directory convention.

6.1 Add-in Structure Requirements and Lifecycle

Create a prefixed subdirectory inside `Addins/` (e.g., `05_my_sensor/`). At startup, `core.addin_loader` scans all subdirectories in alphanumeric order and loads up to three standard files:

- `manifest.json`: Declares the add-in ID, localized names, sidebar icon, sort order, and window geometry.
- `router.py`: Optional FastAPI `APIRouter` module providing custom backend endpoints.
- `window.html`: Frontend user interface rendered inside a dedicated sub-window.

6.2 manifest.json Specification Reference

```
{
  "id": "my_sample_tool",
  "name": "サンプル計測ツール",
  "name_en": "Sample Measurement Tool",
  "description": "Acquires readings from an external sensor and logs them to the
notebook",
  "version": "1.0.0",
  "order": 5,
  "icon": "<svg viewBox=\"0 0 24 24\" fill=\"none\"><circle cx=\"12\" cy=\"12\" r=\"9\"
stroke=\"currentColor\" stroke-width=\"2\"/></svg>",
  "window": {
    "title": "AIN Tool - Sample",
    "width": 600,
    "height": 700,
    "resizable": true
  }
}
```

Note: The `icon` field accepts either an inline SVG markup string or a relative image filename within the same add-in folder (e.g., `"icon": "sensor.png"`).

6.3 Frontend Communication Protocol (ain_addin_bridge)

Use the following standard pattern in your add-in's JavaScript to insert formatted results into the main notebook or read the user's active text selection:

```
// 1. Insert formatted HTML at the active cursor position in the main notebook
const bridge = new BroadcastChannel('ain_addin_bridge');

function sendResultToNotebook(title, value) {
    const html = '<div class="box-wrapper" style="border-left: 4px solid #0d9488;
background-color: #f0fdfa; padding: 8px 12px; margin: 6px 0; font-size: 13px;">'
               + '<b>' + title + ':</b> <span style="font-family: monospace;">' + value +
'</span>'
               + '</div><br>';
    bridge.postMessage({ action: 'insert_html', html: html });
}

// 2. Request and receive the currently selected text from the main notebook
bridge.postMessage({ action: 'request_selection' });
bridge.onmessage = (event) => {
    if (event.data.action === 'response_selection') {
        console.log("Selected notebook text:", event.data.text);
    }
};
```

6.4 Backend API Router Specification (router.py)

To implement hardware communication or custom backend logic, define `router = APIRouter()` inside `router.py`. AIN2 automatically mounts all routes under the prefix `/api/addin/{folder_name}/...`.

```
from fastapi import APIRouter
from pydantic import BaseModel

router = APIRouter()

class SensorPayload(BaseModel):
    channel: int

@router.post("/read_value")
def read_sensor_data(payload: SensorPayload):
    # Perform serial I/O or scientific computation
    measured_val = 123.456
    return {"status": "ok", "channel": payload.channel, "value": measured_val}
```

6.5 Minimal Add-in Implementation Tutorial

The simplest valid add-in requires only `manifest.json` and `window.html`. Duplicating `01_links` and updating the `id` and `name` fields in `manifest.json` immediately registers a new tool icon in the sidebar—enabling standalone plugin development and distribution without modifying a single line of core Python code.

> **Note on Packaging and Distribution**
>
> Any completed add-in folder can be distributed directly as a ZIP archive. End users simply extract the folder into their AIN2 `Addins/` directory and restart the application to enable the new extension.